01 / The next milestone

Purpose

Finish the Step 5 job test on a separate Windows laptop, record defensible evidence, then hand off Step 6 installer development. This guide is for Kushagra, Vaibhav, Nikhil, Chinmay and their agents. Snapshot: 10 October 2026. It documents work and procedures; it does not execute a test, deploy services or grant computer access.

Current status

Onboarding Steps 1-4 passed local acceptance. Step 5 offline distribution is implemented and locally verified: 84 worker tests, 33 backend tests with zero skips, and 32 workload tests passed. A real local Docker/API job processed ten photos as two tasks, with exact output coverage and two metrics. The user reports approved images passed preflight on a separate laptop. Its approved job ownership/results remain pending.

What this final test proves

The receiving laptop obtains approved workload images without a manual Docker build, gets its own owner approval, executes real assigned work, and returns results accepted by the coordinator. Preflight alone proves the runtime can execute a tiny probe. A COMPLETED job alone does not prove which physical laptop worked; retain worker/task ownership evidence.

People and machines

Coordinator: runs the API, PostgreSQL and dashboard, uploads inputs and validates results. Worker: the separate Windows laptop running the owner app and Docker classification. Workers call the coordinator outbound; they do not share RAM as one large computer. Kushagra owns coordinator/contracts; Vaibhav owns worker/runtime; Nikhil owns inputs/evidence; Chinmay owns owner/dashboard UX.

Source and publication boundary

Repository: https://github.com/dkush2024-dot/E-waste-computer-grid. Local base commit is a14f604. Step 5 code and this guide are local changes at this snapshot; they have not been committed/pushed by this agent. Pulling the older main alone will not obtain the new offline flow. Use the current source ZIP, or a reviewed later commit that contains Step 5. The image archive is ignored by Git and must be transferred separately.

After this gate

Record the separate-laptop result, review/commit/push the implementation and sanitized evidence, then implement Step 6 packaging. Step 6 is planned here, not implemented. Physical reboot and multi-device failure testing remain Step 7. No installer, AWS rollout, classifier-accuracy target or measured energy saving follows from the current results.

02 / Prepare the two computers

Files to send to the worker owner

Send output/handoff/E_Waste_Desktop_Worker_Step5_Source.zip and its .zip.sha256 companion, plus output/runtime/team-offline-v1/images.tar. Extract the source ZIP into a new directory and read READ_ME_FIRST.txt. It contains 14 allowlisted source/manifest files and instructions, not an installer or Python runtime. Never send another laptop's .venv, .env, pairing/node credentials, protected state or .test-data.

Prerequisites and one-time source setup

The worker needs Python 3.12 with Tkinter and running Docker Desktop in Linux-container mode on amd64 hardware. From the extracted ZIP directory, run these three commands separately:
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r worker\requirements.txt
.\.venv\Scripts\python.exe worker\desktop.py
If Python/Docker are already set up, reuse a suitable local environment rather than copying one from another machine. Dependency setup may need internet once. Step 6 will remove the need for system Python/source.

Coordinator readiness

Use the existing healthy coordinator and its matching administrator session; do not start a second coordinator on the worker. Default development dashboard/API are localhost:5173 and localhost:8080/api. The earlier isolated demo used 15173 and 18080 instead. Confirm the actual configured ports and current LAN address. The coordinator API must be deliberately reachable over the trusted LAN; keep PostgreSQL private. Preserve existing configuration/data.

Address and consent

Enter the coordinator's actual API URL on the worker, for example http://SERVER_IP:8080/api with SERVER_IP replaced. localhost on the worker points to the worker itself. Select trusted-demo HTTP only for a private LAN; deployments use HTTPS. Choose desired workload limits before pairing, including at least 2 GiB RAM for the default image job. A saved identity retains its earlier frozen limits; editing a textbox cannot expand an accepted approval.

Import or use verified cache

Click Import offline workloads and select images.tar. Expect verification progress, import if images are missing, identity/platform checks, then real preflight. A valid cache skips archive transfer and still runs preflight. The success message is Approved images passed real preflight. This does not enroll a node or start a job worker. Cancellation may leave already imported Docker content, but must not report readiness or start work.

Release integrity

Archive SHA-256: 8f64f8996415c264aa7d144518b1338b4d67878d1be1119992f889fe85e0bdbf.
Source ZIP SHA-256 at this snapshot: 41d301fa1a4c3cadf0b86377c5e85e0c7fc0cc83217ec9c813cd313f99edc6b9.
The trusted worker/releases/current.json fixes the image IDs and archive checksum. Obtain it from reviewed source; replacing both archive and manifest defeats that trust check. Get-FileHash can check transferred files. Hashes identify this release, not future regenerated bundles.

03 / Run the separate-laptop test

1. Prepare the worker identity

After preflight, click Prepare connection. For a new laptop, share only the displayed pairing code with the coordinator administrator. If a valid saved identity exists for this grid, the app shows Already approved; verify that it belongs to this physical laptop and its resource policy, then explicitly Start worker. Never copy another device's identity.

2. Invite and approve

On the coordinator dashboard, enter the new code in the invitation form and submit. On the worker laptop, inspect the grid name/address and approved limits in the popup, then Approve and start. Declining or letting the code expire must not start a job worker. No repeated shared enrollment-token command is needed for this owner flow.

3. Make ownership unambiguous

Give the worker a distinctive name and record its node ID. Start the separate worker before submitting the job. If other workers are contributing, they may claim either task; pause them with their owners' consent for this focused rehearsal, or retain task-owner records that clearly show the external node executed the required work. Do not disable/revoke teammates' devices without authorization.

4. Submit the exact batch

On the coordinator, upload the ten pinned photos in workloads/test-data/demo in alphabetical order. Use image_inference, a distinctive job name such as Step 5 separate Windows laptop, chunk size 5, and the default 2 GiB task RAM requirement. Ten inputs divided into groups of five create two tasks, not two laptops. Record the created job ID.

5. Observe real computation

Expect the registered laptop to move from IDLE to WORKING and back to IDLE after the job. WORKING covers transfer, execution and reporting. Confirm it is not paused and the approved RAM meets admission. Download/preflight/import activity is not itself proof of assigned job execution. Capture a sanitized worker view and ownership record, without pairing codes or credentials.

6. Check and download results

Expect COMPLETED, total_tasks 2, completed_tasks 2, failed_tasks 0, items_processed 10, items_failed 0, ten predictions, and no errors. Download the JSON. Each input must appear exactly once in its expected task chunk; check accepted task ownership and one accepted metric per task. A model score is not verified classification accuracy.

7. Stop safely

Use Stop worker or close the worker app after the rehearsal, unless its owner explicitly wants it to keep participating. Closing waits for owned operations. Preserve its encrypted identity/settings for normal reconnect; do not delete state or use Docker-wide cleanup. Do not destroy coordinator database volumes.

04 / Troubleshoot before changing code

API unavailable or no invitation

Check the current coordinator URL/port and whether the API is running. Same Wi-Fi name does not guarantee reachability; client isolation can block LAN traffic. An optional worker health check is Invoke-RestMethod http://SERVER_IP:8080/api/health -TimeoutSec 10, using the real address/port. Do not open router ports or disable the firewall broadly. Code matching is case-sensitive and expiring; request a fresh session when expired.

Docker missing, stopped or wrong mode

Start Docker Desktop yourself and select the supported Linux-container engine. The initial release requires Linux/amd64; Windows containers and ARM releases are not supported. The source app still needs Python/Tkinter. Read the actual error in Diagnostics rather than patching around it or switching silently to native/dummy mode.

Archive mismatch or runtime failure

Ensure images.tar belongs to the trusted current.json and transfer is complete. Compare SHA-256/size and obtain the reviewed release again if needed. Check host and Docker/WSL disk headroom. A successful import is not enough: real inference preflight must succeed under selected limits. Missing-manifest and wrong-image failures must stay failures.

Job stays QUEUED

Verify the worker is approved, actually started, connected and not paused/revoked. Check image_inference capability and approved RAM versus the job's requirement. A saved approval below 2 GiB stays below 2 GiB after restart; arrange fresh owner approval for a larger policy or an explicitly tested compatible job request. Never increase the server-side ceiling just to make a task run.

Job finishes but the intended laptop stayed IDLE

Another eligible node may have claimed the work. Inspect accepted task ownership; a fleet summary alone cannot identify the machine. Repeat with a controlled participant set or sufficiently attributable records. The current dashboard is not a full attempt-history view. Kushagra can capture reviewed, read-only, secret-free ownership evidence.

Stop, close or network failure is slow

An outstanding HTTP/Docker request can take time to return. Pause drains current work; Stop cancels owned work. The lease protocol handles recovery and rejects stale output. Preserve identity and review bounded redacted logs; do not delete credentials to trigger re-enrollment, kill unrelated containers or reset databases.

When to investigate code

Record the exact symptom, stage, version, worker/job IDs and sanitized error first. Vaibhav traces the desktop/agent/container path; Kushagra traces API admission/lease/results. Fix the root cause in a focused change, run relevant failure regressions, and repeat the failed acceptance stage. Existing local tests passing does not make a physical-machine issue imaginary.

05 / Evidence and the completion gate

Verify the actual download on the coordinator

From the full current source checkout, use your working local Python environment:
.\.venv-local\Scripts\python.exe workloads\test-data\verify_job_results.py "C:/path/to/downloaded-job.json" --report "output/onboarding/step5-separate-laptop-coverage.json"
Replace the file path with the actual download. This CLI expects the pinned ten-photo batch with chunk size five. A different dataset needs a deliberately matching expected-name/chunk verifier.

Record these fields

Test date; coordinator and worker names; distinct physical device descriptions; OS/Python/Docker versions; source commit or ZIP checksum; workload release/image IDs; approved CPU/RAM; whether images were imported or cached; worker node ID; job ID; task IDs/owners; result status/counts; coverage report path; failures and recovery actions. Save screenshots only if useful and remove private content.

Copyable report template

Evidence level: user-reported / independently observed
Separate laptop name + node ID: [fill]
OS + Docker + source/release version: [fill]
Approved CPU/RAM: [fill]
Archive imported or cache reused: [fill]
Job ID + accepted task owners: [fill]
Result: COMPLETED; 2 tasks; 10 predictions; 0 errors [confirm]
Downloaded JSON + exact-coverage report: [fill]
Known failures or unverified scope: [fill]

Minimum response to the project lead

Share the worker name, job ID and result status, followed by the sanitized result/coverage evidence when available. Do not share admin/enrollment tokens, pairing secrets, node credentials, encrypted state or .env files. A report may be user-reported; label it accurately instead of implying the agent watched the laptop.

Close Step 5 only when supported

The current source preview obtains the correct offline release, passes real preflight, receives local owner approval, owns accepted work on the separate laptop and returns exactly accounted outputs. Preserve the local test records. Add a new separate-laptop acceptance record and update step5-verification.json with evidence level, versions and final gate status. Keep the previous pending state as dated history rather than pretending it was already verified.

Publish the source handoff

After review, commit/push the Step 5 implementation, trusted manifest, setup documentation and sanitized evidence to the team repository. Do not commit credentials, environments or the ignored large image archive. Transfer that archive separately. Record the new reviewed commit and source ZIP checksum. A hosted CI pass/clean-clone proof is separate from the recorded local regression results.

06 / Step 6 installer handoff

Goal and current boundary

The eventual user installs a Windows package, opens the app, prepares verified workloads and approves participation without system Python, a source checkout or repeated PowerShell startup. Docker/WSL remains an external prerequisite unless the team deliberately designs and validates supported provisioning. No installer, tray or login startup exists in this snapshot.

Owners and sequence

Vaibhav leads packaging/agent lifecycle; Chinmay leads installer/desktop/tray UX. Kushagra reviews compatibility, admission and release contracts; Nikhil maintains real inputs, image versions and clean-device evidence. First finish/record Step 5, freeze its reviewed source/release, then choose and document a Windows packaging/installer approach. This guide does not select a tool without inspecting the packaging constraints.

Critical implementation work

Bundle Python, Tkinter and agent dependencies with all local modules and the trusted release manifest. Define a packaged agent/preflight entry point: DesktopRuntime currently launches sys.executable plus worker/agent.py, which will not automatically work when sys.executable is the frozen GUI. Keep computation in the existing agent and preserve image IDs, protected per-grid identity, quotas, saved pause, instance locks, cancellation and redacted diagnostics.

Owner controls and install lifecycle

Define window Close versus minimize-to-tray versus Exit visibly. Provide Start/Stop/Pause/Resume in the tray if added. Login startup must be opt-in, visible and reversible; it must not bypass first approval, revive a revoked identity or override pause. Upgrades preserve settings and one identity. Uninstall stops only owned worker processes/containers and removes only this app's startup entries; make state retention/deletion explicit.

Required acceptance

Test a supported clean Windows machine with no system Python/project source; absent/stopped Docker diagnostics; actual image preparation/job; close during transfer/preflight/task; reboot with startup off/on; preserved paused state; duplicate launch; upgrade/rollback and uninstall beside unrelated containers. Record release version, checksums, dependencies, devices and actual versus mocked evidence. Code signing and trusted updates are release decisions, not implied accomplishments.

Copyable next-agent brief

Read the current README, docs/WORKER_ONBOARDING.md, docs/OFFLINE_RUNTIME.md and Step 5 evidence before editing. Verify the separate-laptop gate and exact source; older PDFs are historical. Implement Step 6 packaging with a defined frozen agent/preflight path and opt-in owner controls, preserving the existing execution/consent/state contracts. Inspect relevant code first, choose a simple approach with explained trade-offs, run meaningful packaging/clean-machine checks, and document unverified scope. This document does not authorize desktop control, security-setting changes, registry/AWS publication or messages to others.